Лабораторная работа №4

Линейная алгебра

Ко Антон Геннадьевич

2026-10-24

Содержание I

1. Информация

Раздел 1

Информация

Докладчик

- Ко Антон Геннадьевич
- студент
- Российский университет дружбы народов им. П. Лумумбы
- 1132221551@rudn.ru
- https://SenDerMen04.github.io/ru/



1. Цель работы

Основной целью работы является изучение возможностей специализированных пакетов Julia для выполнения и оценки эффективности операций над объектами линейной алгебры.

2. Выполнение лабораторной работы

2.1 Поэлементные операции над многомерными массивами

Для матрицы 4 × 3 рассмотрим поэлементные операции сложения и произведения её элементов (рис. #fig:001):

[1]:
```
a = rand(1:20,(4,3))
```

[1]:
```
4×3 Matrix{Int64}:
  6   8  16
 16  19  12
 11   7  10
  8  17  12
```

[2]:
```
sum(a)
```

[2]:
```
142
```

[3]:
```
sum(a,dims=1)
```

[3]:
```
1×3 Matrix{Int64}:
 41  51  50
```

[4]:
```
sum(a,dims=2)
```

Для работы со средними значениями можно воспользоваться возможностями пакета Statistics (рис. #fig:002):

```
[6]: prod(a,dims=1)
```

```
[6]: 1×3 Matrix{Int64}:
      8448  18088  23040
```

```
[7]: prod(a,dims=2)
```

```
[7]: 4×1 Matrix{Int64}:
       768
      3648
       770
      1632
```

```
[16]: using Statistics

      mean(a)
```

```
[16]: 11.833333333333334
```

```
[17]: mean(a,dims=1)
```

```
[17]: 1×3 Matrix{Float64}:
      10.25  12.75  12.5
```

```
[18]: mean(a,dims=2)
```

```
[18]: 4×1 Matrix{Float64}:
       10.0
       15.666666666666666
        9.333333333333334
       12.333333333333334
```

2.2 Транспонирование, след, ранг, определитель и инверсия матрицы

Для выполнения таких операций над матрицами, как транспонирование, диагонализация, определение следа, ранга, определителя матрицы и т.п. можно воспользоваться библиотекой (пакетом) LinearAlgebra (рис. #fig:003 – #fig:004):

[20]:
```
using LinearAlgebra

b = rand(1:20, (4,4))
```

[20]:
```
4×4 Matrix{Int64}:
  7  12  13  14
  6  15   6   9
  5  15   9   4
 13  13   5   2
```

[21]:
```
transpose(b)
```

[21]:
```
4×4 transpose(::Matrix{Int64}) with eltype Int64:
  7   6   5  13
 12  15  15  13
 13   6   9   5
 14   9   4   2
```

[22]:
```
tr(b)
```

[22]:
```
33
```

[23]:
```
diag(b)
```

[23]:
```
4-element Vector{Int64}:
  7
```

```
[25]: inv(b)
```

```
[25]: 4×4 Matrix{Float64}:
        0.0325318  -0.0384253  -0.0816832   0.108557
       -0.0693069   0.0891089   0.0544554  -0.0247525
        0.0792079  -0.173267    0.116337   -0.00742574
        0.0410184   0.103725   -0.113861   -0.0261669
```

```
[26]: det(b)
```

```
[26]: -8483.999999999998
```

```
[27]: pinv(a)
```

```
[27]: 3×4 Matrix{Float64}:
       -0.0382636   0.05235     0.0929162  -0.078762
       -0.0308431   0.0203266  -0.0707417   0.0797488
        0.0843685  -0.0414079   0.0134576   0.0010352
```

Рисунок 4: Использование библиотеки LinearAlgebra для выполнения определённых операций

2.3 Вычисление нормы векторов и матриц, повороты, вращения

Для вычисления нормы используется LinearAlgebra.norm(x) (рис. #fig:005):

```
[28]: X = [2, 4, -5]
```

```
[28]: 3-element Vector{Int64}:
        2
        4
       -5
```

```
[29]: norm(X)
```

```
[29]: 6.708203932499369
```

```
[30]: p = 1
      norm(X, p)
```

```
[30]: 11.0
```

```
[31]: X = [2, 4, -5];
      Y = [1,-1,3];
      norm(X-Y)
```

```
[31]: 9.486832980505138
```

```
[32]: sqrt(sum((X-Y).^2))
```

```
[32]: 9.486832980505138
```

```
[33]: acos((transpose(X)*Y)/(norm(X)*norm(Y)))
```

Вычислим нормы для двумерной матрицы (рис. #fig:006):

```
[34]: d = [5 -4 2; -1 2 3; -2 1 0]
```

```
[34]: 3×3 Matrix{Int64}:
        5  -4  2
       -1   2  3
       -2   1  0
```

```
[35]: opnorm(d)
```

```
[35]: 7.147682841795258
```

```
[37]: p = 1
      opnorm(d,p)
```

```
[37]: 8.0
```

```
[38]: rot180(d)
```

```
[38]: 3×3 Matrix{Int64}:
       0   1  -2
       3   2  -1
       2  -4   5
```

```
[39]: reverse(d,dims=1)
```

```
[39]: 3×3 Matrix{Int64}:
       -2   1  0
       -1   2  3
        5  -4  2
```

```
[40]: reverse(d,dims=2)
```

```
[40]: 3×3 Matrix{Int64}:
       2  -4   5
```

2.4 Матричное умножение, единичная матрица, скалярное произведение

Выполним примеры матричного умножения, единичной матрицы и скалярного произведения (рис. #fig:007):

```
[41]: A = rand(1:10,(2,3))
```

```
[41]: 2×3 Matrix{Int64}:
       2  9  2
       9  2  7
```

```
[42]: B = rand(1:10,(3,4))
```

```
[42]: 3×4 Matrix{Int64}:
       1  10  2  10
       9   3  7   3
       8   1  2   3
```

```
[43]: A*B
```

```
[43]: 2×4 Matrix{Int64}:
       99   49  71   53
       83  103  46  117
```

```
[44]: Matrix{Int}(I, 3, 3)
```

```
[44]: 3×3 Matrix{Int64}:
       1  0  0
       0  1  0
       0  0  1
```

```
[46]: X = [2, 4, -5]
```

Решение систем линейных алгебраических уравнений (рис. #fig:008):

```
[48]: A = rand(3, 3)
```

```
[48]: 3×3 Matrix{Float64}:
 0.0564327  0.604788  0.177988
 0.558592   0.386237  0.572085
 0.0425626  0.342446  0.109975
```

```
[49]: x = fill(1.0, 3)
```

```
[49]: 3-element Vector{Float64}:
 1.0
 1.0
 1.0
```

```
[50]: b = A*x
```

```
[50]: 3-element Vector{Float64}:
 0.8392089996554108
 1.516913852995296
 0.49498355571421715
```

```
[52]: A\b
```

```
[52]: 3-element Vector{Float64}:
 0.9999999999997671
 0.9999999999999437
 1.0000000000002653
```

Julia позволяет вычислять LU-факторизацию и определяет составной тип факторизации для его хранения (рис. #fig:009):

```
[53]: Alu = lu(A)
```

```
[53]: LU{Float64, Matrix{Float64}, Vector{Int64}}
L factor:
3×3 Matrix{Float64}:
 1.0        0.0       0.0
 0.101027   1.0       0.0
 0.0761963  0.553259  1.0
U factor:
3×3 Matrix{Float64}:
 0.558592  0.386237   0.572085
 0.0       0.565768   0.120192
 0.0       0.0       -0.000113458
```

```
[54]: Alu.P
```

```
[54]: 3×3 Matrix{Float64}:
 0.0  1.0  0.0
 1.0  0.0  0.0
 0.0  0.0  1.0
```

```
[55]: Alu.p
```

```
[55]: 3-element Vector{Int64}:
 2
 1
 3
```

```
[56]: Alu.L
```

```
[56]: 3×3 Matrix{Float64}:
 1.0        0.0       0.0
 0.101027   1.0       0.0
```

Исходная система уравнений может быть решена или с использованием исходной матрицы, или с использованием объекта факторизации (рис. #fig:010):

```
[58]: A\b

[58]: 3-element Vector{Float64}:
       0.9999999999997671
       0.9999999999999437
       1.0000000000002653

[59]: Alu\b

[59]: 3-element Vector{Float64}:
       0.9999999999997671
       0.9999999999999437
       1.0000000000002653

[60]: det(A)

[60]: 3.585642438785824e-5

[61]: det(Alu)

[61]: 3.585642438785824e-5
```

Рисунок 10: Пример решения с использованием исходной матрицы и с использованием объекта факторизации

Julia позволяет вычислять QR-факторизацию и определяет составной тип факторизации для его хранения (рис. #fig:011):

```
[62]: Aqr = qr(A)
```

```
[62]: LinearAlgebra.QRCompactWY{Float64, Matrix{Float64}, Matrix{Float64}}
Q factor: 3×3 LinearAlgebra.QRCompactWYQ{Float64, Matrix{Float64}, Matrix{Float64}}
R factor:
3×3 Matrix{Float64}:
 -0.563046  -0.469684  -0.593712
  0.0        0.641571   0.136242
  0.0        0.0       -9.92609e-5
```

```
[63]: Aqr.Q
```

```
[63]: 3×3 LinearAlgebra.QRCompactWYQ{Float64, Matrix{Float64}, Matrix{Float64}}
```

```
[65]: Aqr.R
```

```
[65]: 3×3 Matrix{Float64}:
 -0.563046  -0.469684  -0.593712
  0.0        0.641571   0.136242
  0.0        0.0       -9.92609e-5
```

```
[66]: Aqr.Q'*Aqr.Q
```

```
[66]: 3×3 Matrix{Float64}:
  1.0          3.33067e-16  -1.66533e-16
  4.44089e-16  1.0           2.77556e-16
 -1.80411e-16  2.22045e-16   1.0
```

Рисунок 11: Пример вычисления QR-факторизации и определение составного типа факторизации для его хранения

Примеры собственной декомпозиции матрицы (рис. #fig:012):

```
[67]: Asym = A + A'
```

```
[67]: 3×3 Matrix{Float64}:
       0.112865  1.16338   0.220551
       1.16338   0.772474  0.914531
       0.220551  0.914531  0.21995
```

```
[68]: AsymEig = eigen(Asym)
```

```
[68]: Eigen{Float64, Float64, Matrix{Float64}, Vector{Float64}}
      values:
      3-element Vector{Float64}:
       -0.930165085042971
       -0.030608025625123325
        2.0660621749058468
      vectors:
      3×3 Matrix{Float64}:
        0.645161  -0.580308  -0.497001
       -0.653475  -0.08204   -0.752489
        0.395901   0.810254  -0.432145
```

```
[69]: AsymEig.values
```

```
[69]: 3-element Vector{Float64}:
       -0.930165085042971
       -0.030608025625123325
        2.0660621749058468
```

```
[70]: AsymEig.vectors
```

```
[70]: 3×3 Matrix{Float64}:
        0.645161  -0.580308  -0.497001
       -0.653475  -0.08204   -0.752489
        0.395901   0.810254  -0.432145
```

Далее рассмотрим примеры работы с матрицами большой размерности и специальной структуры (рис. #fig:013):

```
 -1.51098    -0.813919    0.278574       0.393205    1.51779     1.03215

[73]: Asym = A + A'

[73]: 1000×1000 Matrix{Float64}:
 -2.73491   -2.36928    -0.384919   …  -0.919445    0.840394   -2.56374
 -2.36928    2.41846    -0.637877      -0.829864    1.55635    -0.819791
 -0.384919  -0.637877   -0.83768        0.994793   -1.11775    -0.200186
 -0.488331   3.11814     1.25716       -1.46842    -0.853927    1.04185
 -0.179865   2.03744    -0.164867      -2.12072     1.66975    -1.01279
 -0.980332  -0.920296   -0.0374544  …   0.843412    0.0927098   0.412074
  0.99688    2.23008    -0.146105      0.279339     1.07918     1.0553
 -1.32111    1.48773    -2.70963      -0.89375     -1.06927    -1.21678
  0.675815  -1.06632     3.12006       1.26461     0.508373    0.525549
  1.24108    2.16016    -0.588434     -1.73126     0.689644   -0.554087
  1.4487     1.12211    -1.34688   …   0.262162   -1.61946    -0.459937
  1.32313   -0.330127   -0.913224     -0.695258    0.0927293   0.982694
 -3.01931    0.0104051  -0.0259752     0.234408   -0.563367    0.21261
  ⋮                                  ⋱
  1.18793   -0.0706867  -0.66757      -1.72264     1.07903     1.67751
  0.327645  -1.14652    -1.18014       0.773148    0.167255   -3.0306
  0.165135   2.45052     0.197755   …   1.36724     1.96973     1.61081
  0.41675    1.36049     0.222903      1.41508     1.17311    -0.586314
 -0.347633  -0.317603    1.5932        1.22226     1.1213     -2.32625
  0.719697  -1.16415    -0.0653141     0.694696   -0.0537789  -0.799803
 -0.33325   -2.64013     0.232735     -1.43069    -0.257561    1.53495
 -0.432046   1.88427     1.01337   …   0.485768    1.34467    -0.596089
  3.99271    0.645979   -2.38219      -0.289051   -0.516378   -1.73132
 -0.919445  -0.829864    0.994793     -2.33488     1.29594    -0.0498035
  0.840394   1.55635    -1.11775       1.29594     3.4721      0.488845
 -2.56374   -0.819791   -0.200186     -0.0498035   0.488845    2.0643

[76]: issymmetric(Asym)
```

Пример добавления шума в симметричную матрицу (матрица уже не будет симметричной) (рис. #fig:014):

```
[78]: Asym_noisy = copy(Asym)
      Asym_noisy[1,2] += 5eps()

[78]: -2.3692843432604214

[79]: issymmetric(Asym_noisy)

[79]: false
```

Рисунок 14: Пример добавления шума в симметричную матрицу

В Julia можно объявить структуру матрицы явно, например, используя Diagonal, Triangular, Symmetric, Hermitian, Tridiagonal и SymTridiagonal (рис. #fig:015):

```
[80]: Asym_explicit = Symmetric(Asym_noisy)
```

```
[80]: 1000×1000 Symmetric{Float64, Matrix{Float64}}:
 -2.73491   -2.36928    -0.384919   …  -0.919445    0.840394   -2.56374
 -2.36928    2.41846    -0.637877      -0.829864    1.55635    -0.819791
 -0.384919  -0.637877   -0.83768        0.994793   -1.11775    -0.200186
 -0.488331   3.11814     1.25716       -1.46842    -0.853927    1.04185
 -0.179865   2.03744    -0.164867      -2.12072     1.66975    -1.01279
 -0.980332  -0.920296   -0.0374544  …   0.843412    0.0927098   0.412074
  0.99688    2.23008    -0.146105       0.279339    1.07918     1.0553
 -1.32111    1.48773    -2.70963       -0.89375    -1.06927    -1.21678
  0.675815  -1.06632     3.12006        1.26461     0.508373    0.525549
  1.24108    2.16016    -0.588434      -1.73126     0.689644   -0.554087
  1.4487     1.12211    -1.34688    …   0.262162   -1.61946    -0.459937
  1.32313   -0.330127   -0.913224      -0.695258    0.0927293   0.982694
 -3.01931    0.0104051  -0.0259752      0.234408   -0.563367    0.21261
  ⋮                                  ⋱
  1.18793   -0.0706867  -0.66757       -1.72264     1.07903     1.67751
  0.327645  -1.14652    -1.18014        0.773148    0.167255   -3.0306
  0.165135   2.45052     0.197755   …   1.36724     1.96973     1.61081
  0.41675    1.36049     0.222903       1.41508     1.17311    -0.586314
 -0.347633  -0.317603    1.5932         1.22226     1.1213     -2.32625
  0.719697  -1.16415    -0.0653141      0.694696   -0.0537789  -0.799803
 -0.33325   -2.64013     0.232735      -1.43069    -0.257561    1.53495
 -0.432046   1.88427     1.01337    …   0.485768    1.34467    -0.596089
  3.99271    0.645979   -2.38219       -0.289051   -0.516378   -1.73132
 -0.919445  -0.829864    0.994793      -2.33488     1.29594    -0.0498035
  0.840394   1.55635    -1.11775        1.29594     3.4721      0.488845
 -2.56374   -0.819791   -0.200186      -0.0498035   0.488845    2.0643
```

Далее для оценки эффективности выполнения операций над матрицами большой размерности и специальной структуры воспользуемся пакетом BenchmarkTools (рис. #fig:016):

```
[84]: using BenchmarkTools

      @btime eigvals(Asym);

        81.436 ms (18 allocations: 7.98 MiB)

[86]: @btime eigvals(Asym_noisy);

        650.002 ms (27 allocations: 7.93 MiB)

[87]: @btime eigvals(Asym_explicit);

        95.206 ms (18 allocations: 7.98 MiB)
```

Рисунок 16: Использование пакета BenchmarkTools

Далее рассмотрим примеры работы с разреженными матрицами большой размерности. Использование типов Tridiagonal и SymTridiagonal для хранения трёхдиагональных матриц позволяет работать с потенциально очень большими трёхдиагональными матрицами (рис. #fig:017):

```
[89]: n = 1000000;
      A = SymTridiagonal(randn(n), randn(n-1))
```

```
[89]: 1000000×1000000 SymTridiagonal{Float64, Vector{Float64}}:
  1.16657   -0.827261   ·        …   ·          ·          ·
 -0.827261   0.625667  1.30581       ·          ·          ·
   ·         1.30581   0.854261      ·          ·          ·
   ·          ·        1.02542       ·          ·          ·
   ·          ·         ·            ·          ·          ·
   ·          ·         ·        …   ·          ·          ·
   ·          ·         ·            ·          ·          ·
   ·          ·         ·            ·          ·          ·
   ·          ·         ·            ·          ·          ·
   ·          ·         ·            ·          ·          ·
   ·          ·         ·        …   ·          ·          ·
   ·          ·         ·            ·          ·          ·
   ·          ·         ·            ·          ·          ·
  ⋮                              ⋱
   ·          ·         ·            ·          ·          ·
   ·          ·         ·            ·          ·          ·
   ·          ·         ·        …   ·          ·          ·
   ·          ·         ·            ·          ·          ·
   ·          ·         ·            ·          ·          ·
   ·          ·         ·            ·          ·          ·
   ·          ·         ·            ·          ·          ·
   ·          ·         ·        …   ·          ·          ·
   ·          ·         ·          0.610911     ·          ·
   ·          ·         ·         -0.0806518   -0.764771   ·
   ·          ·         ·         -0.764771    -0.669177  -0.484408
   ·          ·         ·                      -0.484408   2.5236
```

2.6 Общая линейная алгебра

В примере показано, как можно решить систему линейных уравнений с рациональными элементами без преобразования в типы элементов с плавающей запятой (для избежания проблемы с переполнением используем BigInt) (рис. #fig:018):

```
[92]: Arational = Matrix{Rational{BigInt}}(rand(1:10, 3, 3))/10
```

```
[92]: 3×3 Matrix{Rational{BigInt}}:
       1//2   1//5   1
       1//2    1     1
       1//10  2//5  1//2
```

```
[93]: x = fill(1, 3)
```

```
[93]: 3-element Vector{Int64}:
       1
       1
       1
```

```
[94]: b = Arational*x
```

```
[94]: 3-element Vector{Rational{BigInt}}:
       17//10
        5//2
          1
```

```
[95]: Arational\b
```

```
[95]: 3-element Vector{Rational{BigInt}}:
```

2.7 Самостоятельная работа

Выполнение задания «Произведение векторов» (рис. #fig:019):

```
 0     0    3//10

[97]: using LinearAlgebra

      v = [1, 2, 3]

      dot_v = dot(v, v)

[97]: 14

[98]: outer_v = v * v'

[98]: 3×3 Matrix{Int64}:
       1  2  3
       2  4  6
       3  6  9
```

Рисунок 19: Решение задания «Произведение векторов»

Выполнение задания «Системы линейных уравнений» (рис. #fig:020 – #fig:021):

```
catch e
    println("Нет решения")
end

# Система e
A5 = [1 1 ; 2 1; 1 -1]
b5 = [2; 1; 3]
try
    x5 = A5 \ b5
    println("Ответ e): $x5")
catch e
    println("Нет решения")
end

# Система f
A6 = [1 1 ; 2 1; 3 2]
b6 = [2; 1; 3]
try
    x6 = A6 \ b6
    println("Ответ f): $x6")
catch e
    println("Нет решения")
end
```

```
Ответ a): [2.5, -0.5]
Нет решения
Нет решения
Ответ d): [0.5, 0.5]
```

```
    x2 = A2 \ b2
    println("Ответ b): $x2")
catch e
    println("Нет решения")
end

# Система c
A3 = [1 1 1; 1 1 2; 2 2 3]
b3 = [1; 0; 1]
try
    x3 = A3 \ b3
    println("Ответ c): $x3")
catch e
    println("Нет решения")
end

# Система d
A4 = [1 1 1; 1 1 2; 2 2 3]
b4 = [1; 0; 0]
try
    x4 = A4 \ b4
    println("Ответ d): $x4")
catch e
    println("Нет решения")
end
```

```
Ответ a): [2.214285714285715, 0.35714285714285715, -0.5714285714285711]
Ответ b): [-0.5, 2.5, 0.0]
Нет решения
Нет решения
```

Рисунок 21: Решение задания «Системы линейных уравнений»

Выполнение задания «Операции с матрицами» (рис. #fig:022 – #fig:026):

```
[103]: # Задание 3.1.

       # Матрица a)
       A = [1 -2; -2 1]

       eigen_A = eigen(A)
       diag_matrix= Diagonal(eigen_A.values)
```

```
[103]: 2×2 Diagonal{Float64, Vector{Float64}}:
        -1.0    ⋅
         ⋅     3.0
```

```
[104]: # Матрица b)
       B = [1 -2; -2 3]

       eigen_B = eigen(B)
       diag_matrix= Diagonal(eigen_B.values)
```

```
[104]: 2×2 Diagonal{Float64, Vector{Float64}}:
        -0.236068    ⋅
         ⋅          4.23607
```

```
[105]: # Матрица c)
       C = [1 -2 0; -2 1 2; 0 2 0]

       eigen_C = eigen(C)
       diag_matrix= Diagonal(eigen_C.values)
```

```
[105]: 3×3 Diagonal{Float64, Vector{Float64}}:
        -2.14134    ⋅          ⋅
         ⋅         0.515138    ⋅
         ⋅          ⋅         3.6262
```

Рисунок 22: Решение задания «Операции с матрицами»

[109]:
```
# Задание 3.2.

# Исходная матрица

A = [1 -2; -2 1]

eigen_decomp = eigen(A)
P = eigen_decomp.vectors
D = Diagonal(eigen_decomp.values)

D_10 = D.^10

A_10 = P * D_10 * inv(P)

println(A_10)
```

```
[29525.0 -29524.0; -29524.0 29525.0]
```

Рисунок 23: Решение задания «Операции с матрицами»

[110]:
```
# Исходная матрица

A = [5 -2; -2 5]

eigen_decomp = eigen(A)
eigenvalues = eigen_decomp.values
eigenvectors = eigen_decomp.vectors

sqrt_D = Diagonal(sqrt.(eigenvalues))

sqrt_A = eigenvectors * sqrt_D * inv(eigenvectors)

println(sqrt_A)
```

```
[2.188901059316734 -0.45685025174785676; -0.45685025174785676 2.188901059316734]
```

Рисунок 24: Решение задания «Операции с матрицами»

[111]:
```
# Исходная матрица

A = [1 -2; -2 1]

eigen_decomp = eigen(A)
eigenvalues = eigen_decomp.values
eigenvectors = eigen_decomp.vectors

complex_eigenvalues = Complex.(eigenvalues)
cube_root_D = Diagonal(complex_eigenvalues .^ (1/3))

cube_root_A = eigenvectors * cube_root_D * inv(eigenvectors)

println(cube_root_A)
```

```
ComplexF64[0.971124785153704 + 0.4330127018922193im -0.471124785153704 + 0.4330127018922193im; -0.471124785153704 + 0.4330127018922193im 0.9711247851
53704 + 0.4330127018922193im]
```

Рисунок 25: Решение задания «Операции с матрицами»

```
diag_matrix = Diagonal(eigenvalues)

println("\nДиагональная матрица из собственных значений: ")
println(Matrix(diag_matrix))

lower_triangular = LowerTriangular(A)

println("\nНижнедиагональная матрица из A: ")
println(Matrix(lower_triangular))

@btime eigen(A)
@btime Diagonal(eigenvalues)
@btime LowerTriangular(A)
```

```
  4.214 μs (15 allocations: 2.69 KiB)
Собственные значения матрицы A:
[-0.2360679774997897, 4.23606797749979]

Диагональная матрица из собственных значений:
[-0.2360679774997897 0.0; 0.0 4.23606797749979]

Нижнедиагональная матрица из A:
[140 0 0 0 0; 97 106 0 0 0; 74 89 152 0 0; 168 131 144 54 0; 131 36 71 142 36]
  4.200 μs (15 allocations: 2.69 KiB)
  181.286 ns (1 allocation: 16 bytes)
  186.036 ns (1 allocation: 16 bytes)
```

[116]:
```
5×5 LowerTriangular{Int64, Matrix{Int64}}:
 140    ·    ·    ·    ·
  97  106    ·    ·    ·
  74   89  152    ·    ·
 168  131  144   54    ·
 131   36   71  142   36
```

Рисунок 26: Решение задания «Операции с матрицами»

```
end

function check_productivity_via_spectrum(A)
    eigenvalues = eigvals(A)
    all(abs.(eigenvalues) .< 1)
end

matrices = [A1, A2, A3, A4]
for (i, A) in enumerate(matrices)
    println("Matrix A$i:")
    println(A)
    println("Via (E - A)^(-1) ", check_productivity_via_inverse(A))
    println("Via spectrum: ", check_productivity_via_spectrum(A))
    println("-"^30)
end
```

```
Matrix A1:
[1.0 2.0; 3.0 1.0]
Via (E - A)^(-1) false
Via spectrum: false
------------------------------
Matrix A2:
[0.5 1.0; 1.5 0.5]
Via (E - A)^(-1) false
Via spectrum: false
------------------------------
Matrix A3:
[0.1 0.2; 0.30000000000000004 0.1]
Via (E - A)^(-1) true
Via spectrum: true
------------------------------
Matrix A4:
[0.1 0.2 0.3; 0.0 0.1 0.2; 0.0 0.1 0.3]
Via (E - A)^(-1) true
Via spectrum: true
------------------------------
```

3. Вывод

В ходе выполнения лабораторной работы были изучены возможности специализированных пакетов Julia для выполнения и оценки эффективности операций над объектами линейной алгебры.